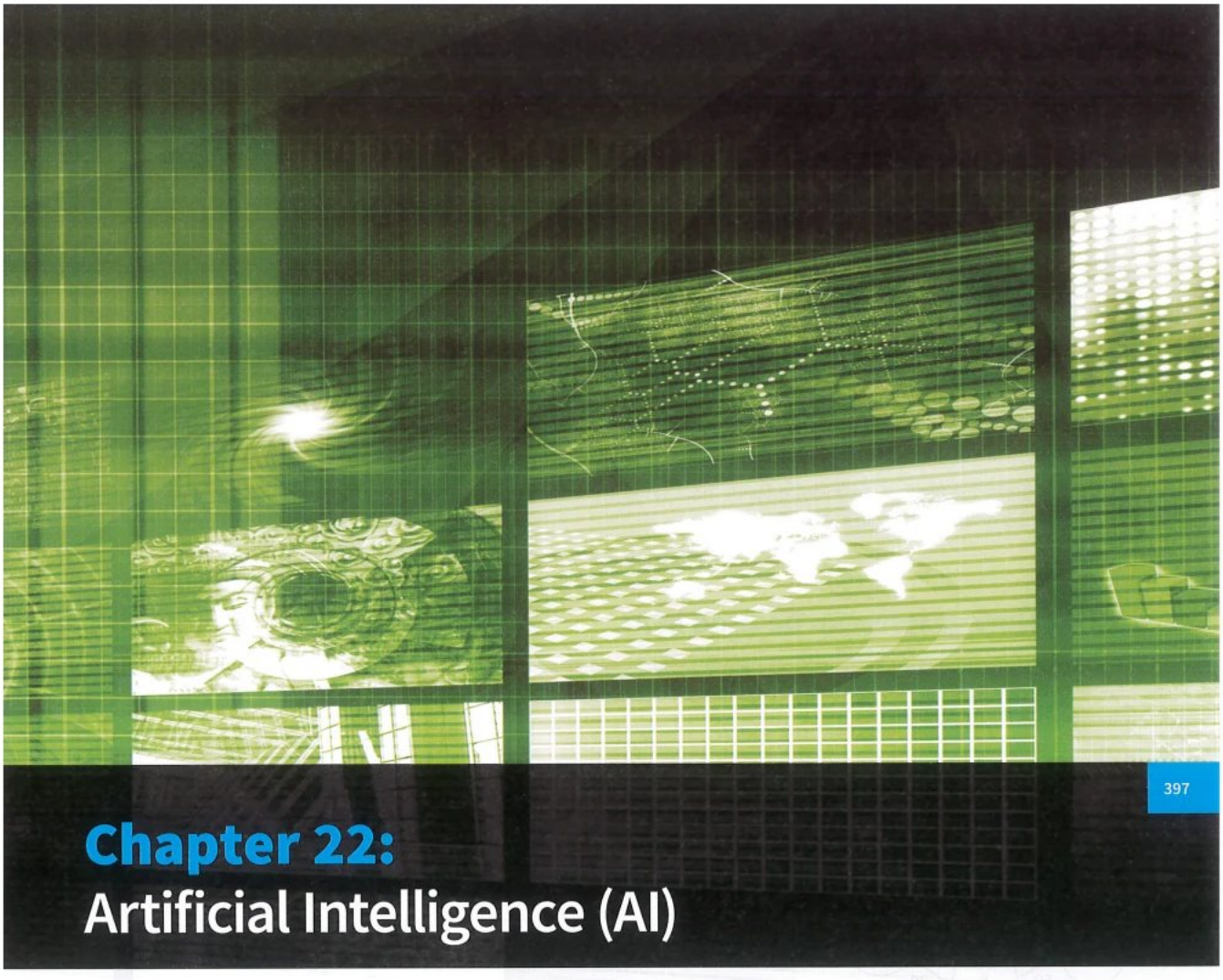
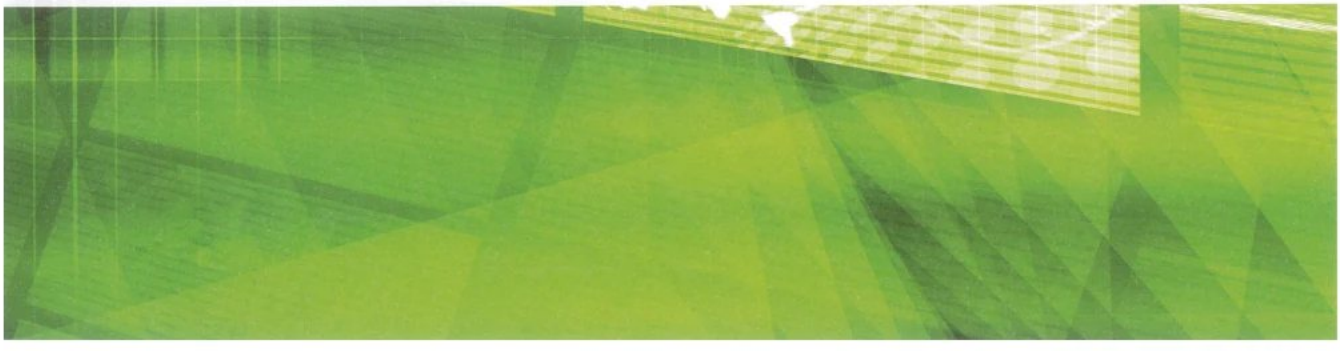


Chapter 22:
Artificial Intelligence (AI)

Learning objectives

By the end of this chapter you should be able to:

- show understanding of how graphs can be used to aid Artificial Intelligence (AI)
- show understanding of how artificial neural networks have helped with machine learning
- show understanding of Deep Learning, Machine Learning and Reinforcement Learning and the reasons for using these methods
- show understanding of back propagation of errors and regression methods in machine learning.



22.01 An overview

It is not easy to define 'Artificial Intelligence'. A key issue is the definition of intelligence. For example, you could argue that a person needs intelligence to do mental arithmetic, such as 43 × 13. You could use a calculator to get the answer, though, and would not describe the calculator as having artificial intelligence. This means that a definition such as:

> Artificial intelligence involves the automation of intelligent behaviour.

is not entirely satisfactory.

There is agreement that AI is a part of computer science. It is also clear that the subject has many distinct sub-sections some of which will be considered in this chapter. The conclusion is that a vague definition is best. For example:

> Artificial Intelligence is concerned with "how to make computers do things at which, at the moment, people are better."
>
> (E. Rich. Artificial Intelligence. McGraw-Hill, 1983)

22.02 How graphs can be used in AI

A graph is a collection of nodes or vertices between which there can be edges. Each node has a name. An edge can have an associated label which is a numerical value. An example is presented in Figure 22.01.

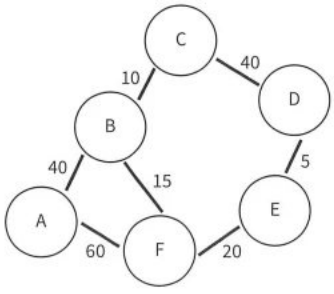


Figure 22.01 An example of a graph with labelled edges

A graph can be used to represent a variety of scenarios. One common representation is that the nodes represent places and the edge labels represent the distances between those places. Edges are only included in the graph when there is a route available for direct travel between the pair of nodes. Such graphs can, for example, find the shortest route between two nodes that are not adjacent to each other.

We could use our intelligence to find the shortest route between node A and node D by considering all of the possible routes and calculating the overall distance for each route. Using Figure 20.01, we would calculate the following values:

For A to B to C to D	overall distance is 40 + 10 + 40 = 90
For A to B to F to E to D	overall distance is 40 + 15 + 20 + 5 = 80, which is the shortest

For A to F to E to D	overall distance is 60 + 20 + 5 = 85
For A to F to B to C to D	overall distance is 60 + 15 + 10 + 40 = 125

For a graph containing 100 nodes this could be quite time consuming. Fortunately, a number of artificial intelligence algorithms have been developed to solve this type of problem.

Question 22.01

Can you think of two alternatives for what the graph in Figure 22.01 might represent? For each of these, state what the values shown as edge labels would represent.

Dijkstra's algorithm

This algorithm finds the shortest path to each of the other nodes starting from one of the nodes.

The following is a Structured English design for a simplified version of the algorithm:

Identify the source node (S) where the path starts.

Create an empty set called the ShortestPath set.

Create another set called RemainingNodes and put all of the nodes into this including the source node (S).

Create a record that stores:

- node names
- calculated values for the distance to the node from the source node
- the sequence of nodes in the route to the node.

Set the distance value for the source node S to be 0.

Set the distance value for all other nodes to be INFINITY where this is to be set as a large value greater than any value that will be calculated.

While the ShortestPath set does not include all of the nodes do the following:

- Pick the node (N) from the RemainingNodes set that has the lowest distance value.
- Move this node into the ShortestPath set.
- For each node in the RemainingNodes set that is adjacent to N:
  - Calculate a new distance value by adding the value given by the label of the edge connecting the two nodes to the already stored distance for N.
  - If this value is less than the value currently stored replace this stored value by the new one that has been calculated.
  - If a new value has been stored enter the sequence of nodes used to obtain this value.

Table 22.01 shows how the algorithm progresses for the graph shown in Figure 22.01. The node names are presented in red when they are still in the RemainingNodes set. At each stage the node N is represented in black. The distance and route data is presented in grey when it is no longer changeable because the node has been moved to the ShortestPath set.

Content of the ShortestPath set	Content of the record					
{}	A	B	C	D	E	F
	0	∞	∞	∞	∞	∞
{A}	A	B	C	D	E	F
	0	40	∞	∞	∞	60
	A	A-B				A-F
{A,B}	A	B	C	D	E	F
	0	40	50	∞	∞	55
	A	A-B	A-B-C			A-B-F
{A,B,C}	A	B	C	D	E	F
	0	40	50	90	∞	55
	A	A-B	A-B-C	A-B-C-D		A-B-F
{A,B,C,F}	A	B	C	D	E	F
	0	40	50	90	75	55
	A	A-B	A-B-C	A-B-C-D	A-B-F-E	A-B-F
{A,B,C,E,F}	A	B	C	D	E	F
	0	40	50	80	75	55
	A	A-B	A-B-C	A-B-F-E-D	A-B-F-E	A-B-F
{A,B,C,D,E,F}	A	B	C	D	E	F
	0	40	50	80	75	55
	A	A-B	A-B-C	A-B-F-E-D	A-B-F-E	A-B-F

Table 22.01 Dijkstra's algorithm applied to the graph in Figure 22.01

Note the general feature that some data don't change once defined as is the case for nodes C and E, but in other cases such as for nodes D and F the data has to be changed at a later stage.

A* algorithm

Dijkstra's algorithm finds the best route from one node to all of the remaining nodes. In many cases the only requirement is to find the best route from one node to just one other node. It would be possible to modify the Dijkstra algorithm to make it stop once the optimum route to the target destination node had been established. However, the modified algorithm would still be likely to carry out far more calculations than were necessary. In particular, the algorithm could initially be exploring routes that were not in the right direction.

The A* algorithm is a modification of the Dijkstra algorithm designed to improve matters. The design of the A* algorithm is different in that the following step:

> Calculate a new distance value by adding the value given by the label of the edge connecting the two nodes to the already stored distance for N.

is expanded to the following:

> Calculate a new distance value by adding the value given by the label of the edge connecting the two nodes to the already stored distance for N.
>
> Calculate an estimated value for the distance of N from the destination node and add this to the new distance value.

The extra calculation of the distance still to be travelled requires the use of a heuristic function. When this function is chosen it must guarantee that any estimated value will be lower than the actual value.

As an example of the use of the A* algorithm we can consider a simple example of finding the shortest route between two towns using the existing roads. Figure 22.02 shows a basic map showing the locations of seven towns identified as A – G.

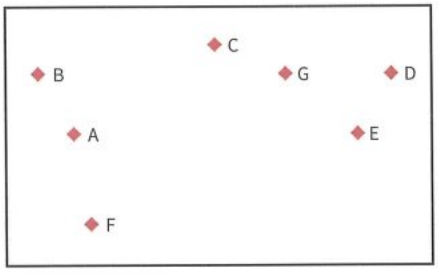


Figure 22.02 A simple map showing the positions of seven towns.

This map is included because it has been drawn by using a pair of x-y coordinates defining the position of each town. These coordinates are the basis of the heuristic function to be used.

Pythagoras's theorem can be used to calculate the direct distance between two positions as follows:

$$\text{Direct distance} = \sqrt{((\text{difference in x coordinates})^2 + (\text{difference in y coordinates})^2)}$$

This calculated distance must be less than any actual distance travelled using existing roads. The coordinates used are shown in Table 22.02.

A	B	C	D	E	F	G
40,20	30,30	80,35	130,30	120,20	45,5	100,30

Table 22.02 The x-y coordinates for the towns A – G

The graph for this problem is shown in Figure 22.03. This is drawn as before using edges to define which towns have an existing road linking the town to another town. The label for each edge shows the distance of travel along that road.

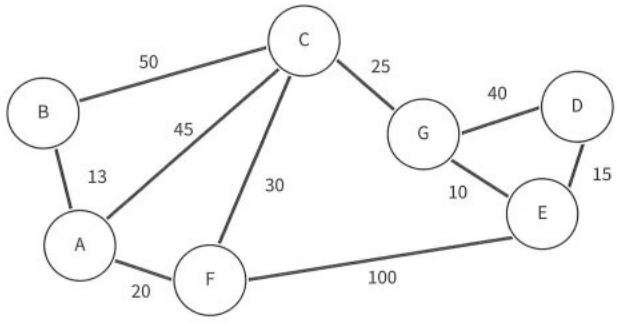


Figure 22.03 The graph used to illustrate the A* algorithm.

It must be understood that there are a variety of versions of the A* algorithm. The application of the algorithm demonstrated here does not match any specific version found in the literature.

The algorithm is documented by the following Structured English.

Create three lists: the initial list, the open list and the closed list.
Insert an empty record for each node in the graph into the initial list.
Store the x and y coordinates of the node in each record of the initial list.
Initialise the value for distance travelled to zero in each record of the initial list.
Create a look-up table that includes an entry for each pair of nodes that have a direct connecting road.
Store the distance of travel along that road for each pair of nodes in the table.
Identify the target node for the travel.
Identify the starting node for the travel and copy the record for this node into the open list.
Delete the record for the starting node from the initial list.

Now recursively apply the following algorithm until all possibilities have been examined:

For the parent node in the open list identify all adjacent nodes in the initial list.
Copy the record for each of the adjacent nodes into the open list.
Delete the records for the adjacent nodes from the initial list.
Check if the target node is now in the open list.

If the target node is not in the open list continue with the following:

> For each adjacent node now added to the open list use the value in the look-up table to determine the distance of travel from the parent node.
> If options exist calculate a value for each route and choose the smallest value.
> For each adjacent node update the value stored for distance travelled in the record .
> For each adjacent node update the path sequence stored in the record.
> For each adjacent node use the heuristic function to calculate a value for estimated distance to travel.
> For each adjacent node calculate a value for estimated total distance from start node to target node by adding the value for distance travelled to the value for estimated distance to travel.
> Identify the adjacent node with the lowest value for total distance from start mode to target node. Leave the record for this node in the open list as a parent node. Copy the records for the remaining nodes into the closed list. Delete these records from the open list.
> Continue with the next iteration of the recursion.

If the target node is in the open list, do the following for this node only:

> Use the value in the look-up table to find the actual distance from the parent node to the target node.
> Calculate the total distance travelled.
> If the record for the target node has zero for the value for distance travelled enter the value now calculated and enter the path sequence into the record.
> If the record already has a value for distance travelled compare the new value and if this indicates a shorter route update the value in the record for distance travelled and enter the new path sequence.
> Copy the record for the target node into the initial list.
> Delete the record for the target node from the open list.
> If there is another adjacent node in the open list continue with the actions listed above for when the target node is not in the open list.
> If the open list is now empty rewind to check nodes in the closed list that have not been included in previous path sequences. For each of these calculate a value for